Requests Library — Full Python Notes

1. What Is the Requests Library?

`requests` is a popular Python library used to send HTTP requests and communicate with web servers and APIs.

It can be used to:

- Send GET, POST, PUT, PATCH, and DELETE requests.
- Fetch data from REST APIs.
- Send JSON data to APIs.
- Submit HTML form data.
- Manage headers, cookies, and authentication.
- Handle HTTP errors and timeouts.
- Download files from the internet.
- Work with sessions and persistent connections.
- Perform basic web scraping.

Important: Requests is a synchronous HTTP client. It does not provide native `async/await` support. For asynchronous HTTP requests, consider HTTPX or aiohttp.

What Is an HTTP Request?

An HTTP request is a message sent by a client to a server to retrieve information or perform an operation.

Example:

```
Python Program → HTTP Request → Web Server
Python Program ← HTTP Response ← Web Server
```

2. Installation

Install the Requests library using pip:

```
pip install requests
```

Check the installed version:

```
import requests

print(requests.__version__)
```

3. Basic GET Request

A GET request retrieves data from a server.

```
import requests

url = "https://httpbin.org/get"

response = requests.get(url)

print(response.status_code)
print(response.text)
```

Explanation

- `requests.get()` sends a GET request.
- `response.status_code` returns the HTTP status code.
- `response.text` returns the response body as text.
- `response.headers` contains the response headers.
- `response.url` returns the final response URL.

Common HTTP Status Codes

Code	Meaning
200	OK
201	Created
204	No Content
301	Moved Permanently
400	Bad Request

401	Unauthorized
403	Forbidden
404	Not Found
429	Too Many Requests
500	Internal Server Error
503	Service Unavailable

4. JSON Response

Many REST APIs return data in JSON format.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.get(url, timeout=10)
response.raise_for_status()

data = response.json()

print(data)
print(data["id"])
print(data["title"])
print(data["body"])
```

Explanation

- `response.json()` parses a JSON response into Python objects.
- JSON objects usually become Python dictionaries.

- JSON arrays usually become Python lists.
- `raise_for_status()` raises an exception for HTTP 4xx and 5xx responses.

Note: A successful HTTP status does not guarantee that the response contains valid JSON. Invalid JSON can raise `requests.exceptions.JSONDecodeError`.

5. GET Request with Query Parameters

Query parameters allow you to request specific information from a server.

```
import requests

url = "https://httpbin.org/get"

params = {
    "name": "Faruk",
    "age": 25,
    "language": "Python",
}

response = requests.get(
    url,
    params=params,
    timeout=10,
)

response.raise_for_status()

print(response.url)
print(response.json())
```

Requests automatically encodes the query parameters and appends them to the URL.

Example URL:

```
https://httpbin.org/get?name=Faruk&age=25&language=Python
```

6. POST Request with JSON

A POST request is commonly used to submit data to a server.

```
import requests

url = "https://httpbin.org/post"
```

```
payload = {
    "name": "Faruk",
    "language": "Python",
    "role": "Backend Developer",
}

response = requests.post(
    url,
    json=payload,
    timeout=10,
)

response.raise_for_status()

print(response.status_code)
print(response.json())
```

Explanation

- `json=payload` serializes the Python object into JSON.
- Requests sets the appropriate JSON content type.
- `response.json()` parses the server's JSON response.

7. POST Request with Form Data

Use `data=` to submit form-encoded data.

```
import requests

url = "https://httpbin.org/post"

data = {
    "username": "faruk",
    "email": "faruk@example.com",
}

response = requests.post(
    url,
    data=data,
    timeout=10,
)
```

```
response.raise_for_status()

print(response.status_code)
print(response.json())
```

Difference Between `data=` and `json=`

Feature	`data=`	`json=`
Typical use	Form data	JSON API payload
Default encoding	Form URL-encoded for a dictionary	JSON
Content type	`application/x-www-form-urlencoded`	`application/json`
Common use case	HTML forms	REST APIs

8. Custom Headers

Headers provide additional information about an HTTP request.

```
import requests

url = "https://httpbin.org/headers"

headers = {
    "User-Agent": "MyPythonApp/1.0",
```

```
    "Accept": "application/json",
}

response = requests.get(
    url,
    headers=headers,
    timeout=10,
)

response.raise_for_status()

print(response.json())
```

Authorization Header

Many APIs use a Bearer token for authentication.

```
import os
import requests

token = os.getenv("API_TOKEN")

if not token:
    raise RuntimeError("API_TOKEN is not configured")

headers = {
    "Authorization": f"Bearer {token}",
    "Accept": "application/json",
}

response = requests.get(
    "https://api.example.com/users",
    headers=headers,
    timeout=10,
)

response.raise_for_status()

print(response.json())
```

Set the environment variable before running the script.

Never commit API tokens or passwords to source control.

9. PUT, PATCH, and DELETE Requests

PUT Request

PUT is commonly used to replace a resource or its representation.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

payload = {
    "id": 1,
    "title": "Updated title",
    "body": "Updated body",
    "userId": 1,
}

response = requests.put(
    url,
    json=payload,
    timeout=10,
)

print(response.status_code)
```

PATCH Request

PATCH is used to partially update a resource.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

payload = {
    "title": "New title",
}

response = requests.patch(
    url,
    json=payload,
    timeout=10,
)
```

```
print(response.status_code)
```

DELETE Request

DELETE requests the removal of a resource.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

response = requests.delete(
    url,
    timeout=10,
)

print(response.status_code)
```

Note: JSONPlaceholder is a testing API. Its responses do not necessarily mean that a real database was modified.

10. Response Object

The response object contains information returned by the server.

```
import requests

response = requests.get(
    "https://httpbin.org/get",
    timeout=10,
)

print(response.status_code)
print(response.reason)
print(response.url)
print(response.headers)
print(response.text)
print(response.content)
print(response.ok)
print(response.history)
```

Important Response Attributes

Attribute	Description
`status_code`	HTTP status code
`reason`	Reason phrase
`url`	Final response URL
`headers`	Response headers
`text`	Response body as text
`content`	Response body as bytes
`encoding`	Text encoding
`cookies`	Cookies received
`history`	Previous responses from redirects

`ok`	True when status code is below 400

Important: `response.ok` does not mean that the status code is exactly 200. For example, redirects can also have `ok=True`.

11. Timeout Handling

Timeouts prevent requests from waiting indefinitely.

```
import requests

try:
    response = requests.get(
        "https://httpbin.org/delay/5",
        timeout=3,
    )

    response.raise_for_status()
    print(response.status_code)

except requests.exceptions.Timeout:
    print("Request timed out")
```

Connect Timeout and Read Timeout

You can configure separate connect and read timeouts.

```
import requests

response = requests.get(
    "https://httpbin.org/get",
    timeout=(3, 10),
)

print(response.status_code)
```

Here:

- `3` is the connection timeout.

- `10` is the read timeout.

Important: Requests has no timeout by default. Also, the read timeout is generally the maximum wait between received bytes, not a strict total wall-clock deadline for the entire request.

12. Error Handling

Use exception handling to manage HTTP errors and network problems.

```
import requests

url = "https://jsonplaceholder.typicode.com/posts/1"

try:
    response = requests.get(
        url,
        timeout=10,
    )

    response.raise_for_status()

    data = response.json()

    print(data)

except requests.exceptions.Timeout:
    print("Request timed out")

except requests.exceptions.HTTPError as error:
    print("HTTP error:", error)

except requests.exceptions.ConnectionError:
    print("Could not connect to the server")

except requests.exceptions.RequestException as error:
    print("Request error:", error)
```

Important Exceptions

Exception	Meaning
`Timeout`	A timeout occurred
`HTTPError`	An unsuccessful HTTP status was raised by `raise_for_status()`
`ConnectionError`	A connection problem occurred
`TooManyRedirects`	Too many redirects occurred
`RequestException`	Base exception for Requests errors

Note: `raise_for_status()` raises `HTTPError` for unsuccessful HTTP status codes. It does not automatically raise an exception for every application-level error.

13. Download a File

Use `response.content` to access binary data.

```
import requests

url = "https://example.com/file.pdf"

try:
    response = requests.get(
        url,
        timeout=30,
    )

    response.raise_for_status()

    with open("file.pdf", "wb") as file:
        file.write(response.content)

    print("File downloaded successfully")

except requests.exceptions.RequestException as error:
    print("Download failed:", error)
```

Download Large Files with Streaming

For large files, use `stream=True` to avoid loading the entire response into memory.

```
import requests

url = "https://example.com/large-file.zip"

try:
    with requests.get(
        url,
        stream=True,
        timeout=(5, 30),
    ) as response:

        response.raise_for_status()

        with open("large-file.zip", "wb") as file:
            for chunk in response.iter_content(
                chunk_size=8192
```

```
                ):
                    if chunk:
                        file.write(chunk)

        print("Download completed")

    except requests.exceptions.RequestException as error:
        print("Download failed:", error)
```

The `with` statement helps close the response and release its connection resources.

14. Upload a File

Use `files=` to upload a file using multipart form data.

```
import requests

url = "https://httpbin.org/post"

try:
    with open("example.txt", "rb") as file:
        response = requests.post(
            url,
            files={
                "file": file,
            },
            timeout=30,
        )

    response.raise_for_status()

    print(response.status_code)
    print(response.json())

except requests.exceptions.RequestException as error:
    print("Upload failed:", error)

except OSError as error:
    print("File error:", error)
```

Requests automatically creates the multipart content type and boundary. Do not manually set the multipart `Content-Type` header unless you also correctly construct

the boundary.

15. Cookies

Cookies allow a server to maintain state across requests.

```
import requests

url = "https://httpbin.org/cookies"

cookies = {
    "username": "faruk",
    "theme": "dark",
}

response = requests.get(
    url,
    cookies=cookies,
    timeout=10,
)

response.raise_for_status()

print(response.json())
```

Read Response Cookies

```
import requests

response = requests.get(
    "https://httpbin.org/cookies/set/session_id/12345",
    timeout=10,
)

print(response.cookies)
print(response.url)
```

For more reliable cookie persistence across requests, use a `Session`.

16. Session and Connection Pooling

A `requests.Session` persists cookies and configuration across requests and reuses connections when possible.

```
import requests

with requests.Session() as session:
    session.headers.update({
        "User-Agent": "MyPythonApp/1.0",
        "Accept": "application/json",
    })

    response1 = session.get(
        "https://httpbin.org/get",
        timeout=10,
    )

    response2 = session.get(
        "https://jsonplaceholder.typicode.com/posts/1",
        timeout=10,
    )

    response1.raise_for_status()
    response2.raise_for_status()

    print(response1.status_code)
    print(response2.json())
```

Benefits of Sessions

- Reuses connections where possible.
- Persists cookies across requests.
- Shares common headers and configuration.
- Reduces unnecessary connection setup.

For repeated requests to the same service, a session is generally preferable to creating a new connection for every request.

17. Authentication

Requests supports several common authentication patterns.

Basic Authentication

```
import requests

url = "https://httpbin.org/basic-auth/faruk/secret"
```

```
response = requests.get(
    url,
    auth=("faruk", "secret"),
    timeout=10,
)

print(response.status_code)
```

Basic authentication credentials should only be sent over HTTPS.

Bearer Token Authentication

```
import os
import requests

token = os.getenv("API_TOKEN")

if not token:
    raise RuntimeError("API_TOKEN is not configured")

headers = {
    "Authorization": f"Bearer {token}",
}

response = requests.get(
    "https://api.example.com/profile",
    headers=headers,
    timeout=10,
)

response.raise_for_status()

print(response.json())
```

The API URL is illustrative. Replace it with the actual endpoint provided by your service.

18. Redirect Handling

Requests follows redirects by default for most common HTTP methods.

```
import requests
```

```
response = requests.get(
    "https://httpbin.org/redirect/1",
    timeout=10,
)

print(response.status_code)
print(response.url)
print(response.history)
```

Disable Redirects

```
import requests

response = requests.get(
    "https://httpbin.org/redirect/1",
    allow_redirects=False,
    timeout=10,
)

print(response.status_code)
print(response.headers.get("Location"))
```

Use `allow_redirects=False` when you need to inspect the original redirect response.

19. Retry Failed Requests

Temporary connection failures and certain server errors may be recoverable.

Use `HTTPAdapter` and `Retry` to configure retries for appropriate requests.

```
import requests
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

retry_strategy = Retry(
    total=3,
    connect=3,
    read=0,
    status=3,
    backoff_factor=0.5,
    status_forcelist=[429, 500, 502, 503, 504],
    allowed_methods=frozenset([
```

```
            "GET",
            "HEAD",
            "OPTIONS",
        ]),
        respect_retry_after_header=True,
    )

    adapter = HTTPAdapter(
        max_retries=retry_strategy
    )

    with requests.Session() as session:
        session.mount("https://", adapter)
        session.mount("http://", adapter)

        response = session.get(
            "https://httpbin.org/status/503",
            timeout=10,
        )

        print(response.status_code)
```

Important Retry Rules

- Limit the number of retries.
- Respect API rate limits and `Retry-After`.
- Use exponential backoff where appropriate.
- Be cautious when retrying POST, PATCH, or other non-idempotent operations.
- Retry only errors that are likely to be temporary.

The example uses `read=0` to avoid automatically retrying read failures, which can be important when the server may have processed a request already.

20. Working with APIs

A reusable function can make API requests easier to maintain.

```
import requests

BASE_URL = "https://jsonplaceholder.typicode.com"


def get_post(post_id):
```

```
    url = f"{BASE_URL}/posts/{post_id}"

    response = requests.get(
        url,
        timeout=10,
    )

    response.raise_for_status()

    return response.json()


try:
    post = get_post(1)

    print("ID:", post["id"])
    print("Title:", post["title"])
    print("Body:", post["body"])

except requests.exceptions.RequestException as error:
    print("API request failed:", error)
```

21. Synchronous Requests vs Asynchronous Requests

Requests is synchronous. A request generally blocks the current thread until the operation completes or fails.

```
import requests

response = requests.get(
    "https://jsonplaceholder.typicode.com/posts/1",
    timeout=10,
)

print(response.json())
```

For multiple independent requests, synchronous execution often waits for one request to finish before moving to the next.

```
Request 1 → Wait → Complete
Request 2 → Wait → Complete
Request 3 → Wait → Complete
```

When Do You Need Async HTTP?

If you need to handle many independent network requests concurrently, consider:

- HTTPX with `AsyncClient`.
- aiohttp with `ClientSession`.
- A thread pool with Requests for suitable synchronous workloads.

Do not call blocking Requests operations directly inside an asynchronous event loop if the loop must remain responsive.

22. Security Best Practices

1. Always configure appropriate timeouts.
2. Use HTTPS for sensitive data.
3. Keep API tokens out of source code.
4. Avoid logging passwords, tokens, or sensitive response data.
5. Keep TLS certificate verification enabled.
6. Handle errors without exposing secrets.
7. Respect API rate limits and terms of service.
8. Validate API response data before using it.
9. Use retries carefully to avoid duplicate operations.
10. Avoid sending credentials to untrusted URLs.

TLS Certificate Verification

Requests verifies TLS certificates by default.

```
import requests

response = requests.get(
    "https://example.com",
    timeout=10,
    verify=True,
)

print(response.status_code)
```

Avoid `verify=False` in production because it disables certificate verification and exposes the connection to interception risks.

23. Common Mistakes

1. Forgetting to install `requests`.

2. Forgetting to configure timeouts.
3. Using `response.json()` when the response is not valid JSON.
4. Forgetting `raise_for_status()`.
5. Assuming every HTTP 200 response contains the expected data.
6. Using `data=` when the API expects JSON.
7. Creating new sessions unnecessarily for repeated requests.
8. Loading large downloads entirely into memory.
9. Hardcoding API credentials.
10. Disabling TLS certificate verification.
11. Retrying non-idempotent requests without considering duplicate operations.
12. Expecting Requests to support `await` or native asynchronous requests.

24. Requests vs HTTPX vs aiohttp

Feature	Requests	HTTPX	aiohttp
Synchronous client	Yes	Yes	Not its main client API
Asynchronous client	No native async client	Yes	Yes
`async/await`	No	Yes	Yes
Connection pooling	Yes, with Session	Yes	Yes
Timeout configuration	Yes	Yes	Yes
HTTP/2 client	No	Yes, optional	Not a standard

support			client feature
Best known for	Simple synchronous HTTP	Sync and async HTTP	Async networking and server support

Which One Should You Learn?

Requests: Learn this first for basic HTTP requests, REST API integration, scripts, and simple synchronous tasks.

HTTPX: Learn this when you need both synchronous and asynchronous clients or optional HTTP/2 support.

aiohttp: Learn this when your project requires asynchronous networking or its asynchronous web server.

25. Interview Questions

Q1. What is the Requests library?

Requests is a Python library that simplifies sending HTTP requests and handling HTTP responses.

Q2. What is the difference between GET and POST?

GET is generally used to retrieve resources, while POST is commonly used to submit data or trigger server-side processing.

Q3. What is the difference between `response.text` and `response.content`?

`response.text` returns the body decoded as text. `response.content` returns the body as bytes.

Q4. What is `response.json()`?

It parses a JSON response body into a corresponding Python object, such as a dictionary or list.

Q5. What is `raise_for_status()`?

It raises `HTTPError` when the response status code indicates an HTTP 4xx or 5xx error.

Q6. What is a Session?

A Session persists cookies and configuration across requests and enables connection reuse.

Q7. Why are timeouts important?

Timeouts prevent a request from waiting indefinitely for a connection or response data.

Q8. What is the difference between `data=` and `json=`?

`data=` commonly submits form-encoded data, while `json=` serializes data as JSON.

Q9. Does Requests support asynchronous programming natively?

No. Requests is synchronous. Use HTTPX or aiohttp for native asynchronous HTTP requests.

Q10. What is connection pooling?

Connection pooling reuses existing network connections when possible, reducing the overhead of establishing new connections.

Q11. What is the difference between connect timeout and read timeout?

Connect timeout limits how long Requests waits to establish a connection. Read timeout limits how long it waits between incoming bytes.

Q12. What is the purpose of `stream=True`?

It allows response content to be consumed incrementally rather than automatically loading the entire response body into memory.

26. Practice Projects

1. **API Data Fetcher:** Fetch 10 posts from a public API and save them to a JSON file.
2. **Weather API Client:** Fetch current weather data from a suitable weather API.
3. **File Downloader:** Download files using streaming and timeout handling.
4. **Retry Handler:** Implement limited retries for transient errors.
5. **API Authentication Client:** Send authenticated requests using environment variables.
6. **Paginated API Fetcher:** Retrieve data from an API that returns results page by page.

7. **Rate-Limited Scraper:** Collect information from an authorized website while respecting its rules.
8. **Django API Integration:** Call an external REST API from a Django backend and handle errors.

27. Quick Revision Cheat Sheet

Task	Requests
GET request	`requests.get(url)`
POST request	`requests.post(url, json=data)`
PUT request	`requests.put(url, json=data)`
PATCH request	`requests.patch(url, json=data)`
DELETE request	`requests.delete(url)`

Query parameters	`params=params`
JSON payload	`json=data`
Form data	`data=data`
Custom headers	`headers=headers`
Authentication	`auth=(username, password)`
JSON response	`response.json()`
Text response	`response.text`
Binary response	`response.content`
HTTP error checking	`response.raise_for_status()`

Timeout	`timeout=10`
Connect/read timeout	`timeout=(3, 10)`
Persistent session	`requests.Session()`
Stream download	`stream=True`
Disable redirects	`allow_redirects=False`
TLS verification	`verify=True`

28. Official Documentation

- Requests documentation: https://requests.readthedocs.io/en/latest/
- Quickstart: https://requests.readthedocs.io/en/latest/user/quickstart/
- Advanced Usage: https://requests.readthedocs.io/en/latest/user/advanced/
- API Reference: https://requests.readthedocs.io/en/latest/api/

Learning Recommendation

Follow this order:

1. Understand HTTP methods and status codes.
2. Practice GET and POST requests.
3. Learn JSON responses, query parameters, and headers.
4. Understand timeouts and exception handling.
5. Learn Sessions, cookies, and authentication.

6. Practice downloading and uploading files.
7. Learn retry strategies and API pagination.
8. Build a small API client.
9. Move to HTTPX and aiohttp when you need asynchronous requests.

Final Tip: Requests is an excellent starting point for learning HTTP and REST APIs in Python. Once you understand its fundamentals, learning HTTPX and aiohttp becomes much easier.